

Direct Trajectory Optimization and Sequential Quadratic Programming

Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 22, 2026

Minimum-time formulation:

$$\min_{u(\cdot), T} T$$

subject to

$$\dot{x} = f(x, u)$$

$$x(0) = x_0, \quad x(T) = x_{goal}$$

$$u_{min} \leq u(t) \leq u_{max}$$

Time-Scaling Trick

Discretize with step size h :

$$x_{k+1} = x_k + hf(x_k, u_k)$$

If $T = Nh$ is decision variable:

$$\min_{x, u, h} Nh$$

Now h becomes optimization variable.

Important:

$$h > 0$$

Otherwise solver can make $h \rightarrow \infty$.

Even if f linear $\rightarrow$ problem nonlinear.

Direct Trajectory Optimization

Convert OCP into NLP.

$$\min_z F(z)$$

subject to

$$c(z) = 0 \quad (\text{dynamics})$$

$$d(z) \leq 0 \quad (\text{constraints})$$

Decision variables:

$$z = \{x_0, \dots, x_N, u_0, \dots, u_{N-1}\}$$

Large-scale but structured NLP.

Sequential Quadratic Programming (SQP)

Define Lagrangian:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda^T c(x)$$

Second-order Taylor approximation:

$$\min_{\Delta x} \nabla f^T \Delta x + \frac{1}{2} \Delta x^T H \Delta x$$

subject to

$$c(x) + J_c \Delta x = 0$$

where

$$H \approx \nabla_{xx}^2 \mathcal{L}$$

Solve QP $\rightarrow$ search direction.

Quadratic subproblem KKT:

$$\begin{bmatrix} H & J_c^T \\ J_c & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta \lambda \end{bmatrix} = - \begin{bmatrix} \nabla f + J_c^T \lambda \\ c(x) \end{bmatrix}$$

Newton-like step for constrained problems.

If only equalities $\rightarrow$ exact Newton.

Direct Collocation Idea

Instead of forward simulation:

$$x_{k+1} = f(x_k, u_k)$$

Enforce dynamics as equality constraints:

$$g(x_k, u_k, x_{k+1}) = 0$$

Implicit integration becomes possible.

Hermite-Simpson Collocation

Define midpoint:

$$x_{k+\frac{1}{2}} = \frac{1}{2}(x_k + x_{k+1}) + \frac{h}{8}(f_k - f_{k+1})$$

where

$$f_k = f(x_k, u_k)$$

Midpoint control:

$$u_{k+\frac{1}{2}} = \frac{1}{2}(u_k + u_{k+1})$$

Hermite-Simpson Constraint

Collocation constraint:

$$x_{k+1} = x_k + \frac{h}{6} \left(f_k + 4f_{k+\frac{1}{2}} + f_{k+1} \right)$$

Equivalent constraint form:

$$g_k(x_k, x_{k+1}, u_k, u_{k+1}) = 0$$

Third-order accurate integration.

Why Collocation is Efficient

Explicit RK3:

$$f_1 = f(x_k, u_k)$$

$$f_2 = f\left(x_k + \frac{h}{2}f_1, u_k\right)$$

$$f_3 = f(x_k - hf_1 + 2hf_2, u_k)$$

$$x_{k+1} = x_k + \frac{h}{6}(f_1 + 4f_2 + f_3)$$

Requires 3 dynamics calls.

Hermite-Simpson Advantage

In collocation:

$$f_k, f_{k+1}$$

are reused from adjacent intervals.

Only one new evaluation per interval.

Savings:

$$30\% - 50\%$$

Critical when dynamics expensive.

Comparison of Methods

Shooting	Collocation
Forward simulate	Enforce constraints
Sensitive to initial guess	More stable
Small NLP	Large sparse NLP
Cheap per iter	Better convergence

DIRCOL = Direct Collocation.

- Warm start is critical
- Exploit sparsity in KKT systems
- IPOPT, SNOPT, KNITRO commonly used
- Contact problems $\rightarrow$ highly non-convex

Example:

Acrobot swing-up via DIRCOL.